

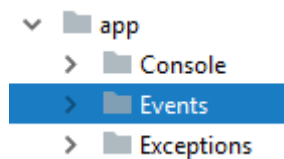
TP Events / Listners

On a un sujet et des observateurs. Les observateurs doivent s'inscrire (on dit aussi s'abonner) auprès du sujet. Lorsque le sujet change d'état il notifie tous ses abonnés. Un observateur peut aussi se désinscrire, il ne recevra alors plus les notifications.

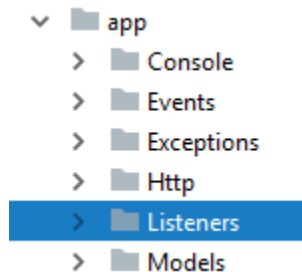
Organisation du code

Laravel implémente ce design pattern et le rend très simple d'utilisation. On va avoir :

- des classes **Event** placées dans le dossier **app/Events** :



- des classes **Listener** placées dans le dossier **app/Listeners** :



Ces dossiers n'existent pas dans l'installation de base, ils sont ajoutés dès qu'on crée la première classe avec Artisan qui dispose de commandes pour le faire :

<code>make:event</code>	Create a new event class
<code>make:exception</code>	Create a new custom exception class
<code>make:factory</code>	Create a new model factory
<code>make:job</code>	Create a new job class
<code>make:listener</code>	Create a new event listener class

Les événements déjà présents dans Laravel

Il n'y a pas que les événements que vous allez créer. Vous pouvez utiliser les événements existants déjà dans le framework qui en propose au niveau de l'authentification :

- Registered
- CurrentDeviceLogout
- OtherDeviceLogout
- Attempting
- Authenticated

- Login
- Failed
- Logout
- Lockout
- PasswordReset
- Validated
- Verified
- ...

On trouve aussi des événements avec Eloquent :

- retrieved
- creating
- created
- updating
- updated
- saving
- saved
- deleting
- deleted
- restoring
- restored
- ...

Pour tous ces cas vous n'avez donc pas à vous inquiéter du déclencheur qui existe déjà.

Une application de test

Pour nos essais créez une nouvelle application LaravelEvents :

```
composer create-project laravel/laravel laravelEvents
```

Créez une base, renseignez correctement le fichier **.env**. Installez Breeze et lancez les migrations :

```
composer require laravel/breeze --dev  
php artisan breeze:install  
npm install  
npm run dev  
php artisan migrate
```

Créez un utilisateur à partir du formulaire d'enregistrement :

Name

Durand

Email

durand@chezlui.fr

Password

••••••••

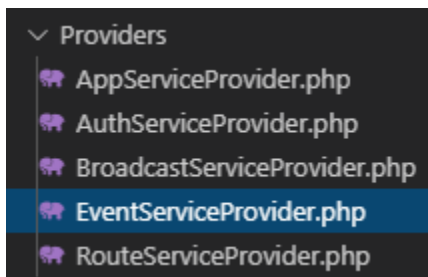
Confirm Password

••••••••

[Already registered?](#) **REGISTER**

Le fournisseur de service (service provider)

Il y a un fournisseur de service dédié aux événements :



Par défaut on a ce code :

```
<?php

namespace App\Providers;

use Illuminate\Auth\Events\Registered;
use Illuminate\Auth\Listeners\SendEmailVerificationNotification;
use Illuminate\Foundation\Support\Providers\EventServiceProvider as ServiceProvider;
use Illuminate\Support\Facades\Event;

class EventServiceProvider extends ServiceProvider
{
    /**
     * The event listener mappings for the application.
     */
}
```

```

* @var array<class-string, array<int, class-string>>
*/
protected $listen = [
    Registered::class => [
        SendEmailVerificationNotification::class,
    ],
];

/**
 * Register any events for your application.
 *
 * @return void
 */
public function boot()
{
    //
}

/**
 * Determine if events and listeners should be automatically discovered.
 *
 * @return bool
 */
public function shouldDiscoverEvents()
{
    return false;
}
}

```

On a une propriété **\$listen** pour les abonnements. C'est un simple tableau qui prend l'événement (**event**) comme clé et l'observateur (**listener**) comme valeur. On voit qu'on a déjà l'événement **Registered** qui renvoie à l'observateur **SendEmailVerificationNotification**. On a vu avec jetstream que lorsqu'un utilisateur s'enregistre on peut lui envoyer un email, mais on ne s'était pas posé la question de savoir comment ça se passait en interne. On a ici une réponse, on observe l'événement **Registered** et on lance le code pour la notification avec le listener.

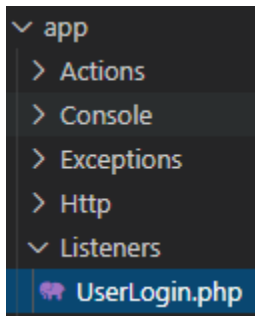
Laravel prévoit de placer les observateurs (**listeners**) dans un dossier **app/Listeners** (qui n'existe pas au départ) en tant que classes, de même pour les événements (**events**) dans le dossier **Events** (qui n'existe pas au départ).

On crée une écoute

On va utiliser l'événement intégré à Laravel qui se déclenche lorsqu'un utilisateur se connecte. La classe correspondante est **Illuminate\Auth\Events\Login**. Puisque l'événement existe on n'a pas à créer une classe **Event**. Par contre il nous faut une écoute (**Listener**) :

```
php artisan make:listener UserLogin
```

La classe **UserLogin** se crée ainsi que le dossier :



Voici le code généré :

```
<?php

namespace App\Listeners;

use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Contracts\Queue\ShouldQueue;

class UserLogin
{
    /**
     * Create the event listener.
     *
     * @return void
     */
    public function __construct()
    {
        //
    }
}
```

```

/**
 * Handle the event.
 *
 * @param object $event
 * @return void
 */
public function handle($event)
{
    //
}
}

```

On va établir le lien entre l'événement et l'écoute dans le provider **EventServiceProvider** :

```

use Illuminate\Auth\Events\Login;
use App\Listeners\UserLogin;

...

protected $listen = [

...

Login::class => [UserLogin::class],

];

```

Donc maintenant dès que l'utilisateur va se connecter la méthode **handle** de notre écoute va être appelée. Mais qu'y a-t-il dans la variable **\$event** ? On va faire un petit test :

```

public function handle($event)
{
    dd($event);
}

```

Là dès qu'on se connecte on obtient :

```

Illuminate\Auth\Events\Login {#399 ▼
  +guard: "web"

  +user: App\Models\User {#1370  }

  +remember: false

```

```
}
```

Une instance de la classe **Illuminate/Auth/Events/Login** avec plusieurs propriétés, en particulier une instance du modèle de l'utilisateur et un booléen pour le **remember**. On aurait aussi pu trouver ces informations en allant voir directement la classe.

Changez ainsi le code :

```
public function handle($event)
{
    dd($event->user->name . " s'est connecté.");
}
```

Cette fois à la connexion on obtient quelque chose dans ce genre :

"Durand s'est connecté."

On peut donc ici effectuer tous les traitements qu'on veut, comme comptabiliser les connexions.

Découverte automatique des événements

On a vu ci-dessus qu'on a créé un listener pour un événement existant dans Laravel (**Login**) et qu'on a dû déclarer le lien entre événement et listener dans le provider **EventServiceProvider**. Mais il est possible de rendre ce lien automatique, pour ça il faut faire deux choses :

- comme Laravel utilise la réflexion il faut un peu l'aider et préciser la classe de l'événement dans la fonction **handle** du listener :

```
use Illuminate\Auth\Events>Login;
...
public function handle(Login $event)
```

- il faut aussi changer la valeur de retour dans cette méthode du provider **EventServiceProvider** :

```
public function shouldDiscoverEvents()
{
    return true;
}
```

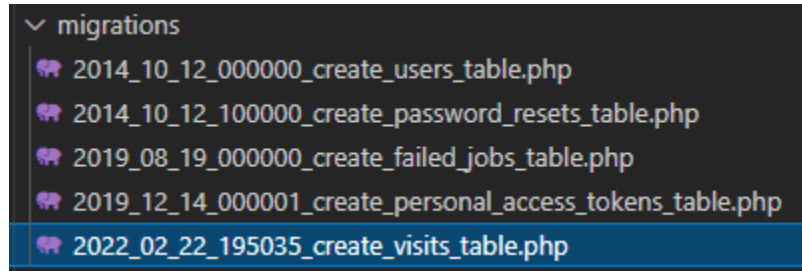
Maintenant on peut s'éviter l'écriture de la liaison dans la propriété **\$listen** !

On crée un événement

Maintenant on va créer un événement et aussi son écoute. On va imaginer qu'on veut savoir quand quelqu'un affiche la page d'accueil et mémoriser son IP ainsi que le moment du chargement de la page, en gros un peu de statistique.

On va commencer par créer une migration :

```
php artisan make:migration create_visits_table --create=visits
```



Avec ces créations de colonnes :

```
public function up()
{
    Schema::create('visits', function (Blueprint $table) {
        $table->id();
        $table->string('ip', 45);
        $table->timestamp('created_at');
    });
}
```

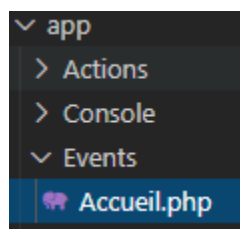
Et on lance la migration :

```
php artisan migrate
```

On crée l'événement :

```
php artisan make:event AccueilEvent
```

On le trouve ici avec création du dossier :



Avec ce code par défaut :

```
<?php
```



```
namespace App\Events;

use Illuminate\Broadcasting\Channel;

use Illuminate\Broadcasting\InteractsWithSockets;

use Illuminate\Broadcasting\PresenceChannel;

use Illuminate\Broadcasting\PrivateChannel;

use Illuminate\Contracts\Broadcasting\ShouldBroadcast;

use Illuminate\Foundation\Events\Dispatchable;

use Illuminate\Queue\SerializesModels;

class AccueilEvent

{

    use Dispatchable, InteractsWithSockets, SerializesModels;

    /**
     * Create a new event instance.
     *
     * @return void
     */

    public function __construct()

    {

        //

    }

    /**
     * Get the channels the event should broadcast on.
     *
     * @return \Illuminate\Broadcasting\Channel|array
     */

    public function broadcastOn()

    {

        return new PrivateChannel('channel-name');

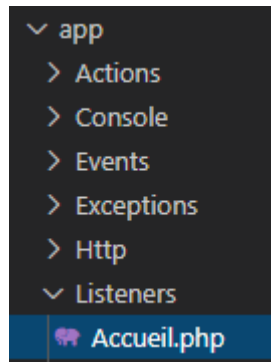
    }

}
```

On ne va pas toucher à ce code.

On crée aussi l'écoute :

```
php artisan make:listener AccueilListner
```



On modifie ainsi le code :

```
<?php

namespace App\Listeners;

use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Support\Facades\DB;
use Carbon\Carbon;
use App\Events\Accueil as AccueilEvent;

class AccueilListner
{
    public function __construct()
    {
        //
    }

    public function handle(AccueilEvent $event)
    {
        DB::table('visits')->insert([
            'ip' => request()->ip(),
            'created_at' => Carbon::now(),
        ]);
    }
}
```

```
}
```

J'utilise directement le Query Builder sans passer par Eloquent.

Il ne reste plus qu'à déclencher cet événement dans la route :

```
use App\Events\AccueilEvent;

Route::get('/', function () {
    event(new AccueilEvent);

    return view('welcome');
});
```

Maintenant chaque fois que la page d'accueil est ouverte l'événement **Accueil** est déclenché et donc l'écoute **Accueil** en est informée. On sauvegarde alors dans la base l'IP et le moment du chargement :

id	ip	created_at
1	127.0.0.1	2022-02-22 19:55:31

Si le traitement de vos événements est long (envoi d'email, requête...) il est conseillé d'utiliser un système de file d'attente. Vous pouvez consulter la documentation correspondante.